

DETAILED PROJECT REPORT

Web Vulnerability Scanner based on OWASP Top 10

Submitted By:

Amal S

College of Engineering Chengannur

1. Objectives

- Develop an automated web vulnerability scanner.
- Detect common web vulnerabilities such as Cross-Site Scripting (XSS) and SQL Injection (SQLi).
- Understand OWASP Top 10 security risks.
- Generate vulnerability reports for security assessment.
- Improve awareness of secure web development practices.

2. Benefits

- Automates security testing.
- Helps identify vulnerabilities before attackers exploit them.
- Reduces manual security assessment effort.
- Improves web application security.
- Supports cybersecurity learning and research.

3. Problem Statement

Web applications are increasingly targeted by cyber attackers. Vulnerabilities such as XSS and SQL Injection can lead to data breaches, unauthorized access, and service disruption. Manual testing is time-consuming and error-prone. Therefore, an automated vulnerability scanner is required to identify security weaknesses efficiently.

4. Frame the Problem

4.1 Type of Problem

Cybersecurity Assessment and Vulnerability Detection.

4.2 Nature of Task

Security Testing and Vulnerability Analysis.

4.3 Approach

Automated scanning using Python scripts, form analysis, payload injection, and response validation.

4.4 Performance Measurement

- Number of vulnerabilities detected
- Detection accuracy
- Scan completion time
- False positive rate

5. Hypotheses

User-Level Hypotheses

- Websites with improper input validation are vulnerable to XSS.
- Login forms may be susceptible to SQL Injection.

Application-Level Hypotheses

- Applications lacking sanitization mechanisms have higher security risks.
- Dynamic web forms are more likely to expose vulnerabilities.

6. Data Exploration

The scanner analyzes:

- Website URLs
- Forms
- Input fields
- Server responses
- Error messages

Collected information is used to identify security weaknesses.

7. Strategy: Solving the Problem

Automated Vulnerability Detection

The system automatically crawls web pages and identifies forms.

Payload Injection

Predefined XSS and SQL Injection payloads are submitted to input fields.

Response Analysis

Responses are examined for signs of successful exploitation.

Report Generation

Detected vulnerabilities are stored in a report file.

8. Methodology and Workflow

Step 1: Target URL Input

User provides the website URL.

Step 2: Crawling

The crawler discovers pages and links.

Step 3: Form Detection

The system identifies all forms.

Step 4: Payload Injection

XSS and SQL Injection payloads are inserted.

Step 5: Response Analysis

Responses are checked for vulnerability indicators.

Step 6: Reporting

A report containing findings is generated.

9. Vulnerability Detection

9.1 Cross-Site Scripting (XSS)

XSS allows attackers to inject malicious JavaScript into web pages.

Example:

```
<script>alert('XSS')</script>
```

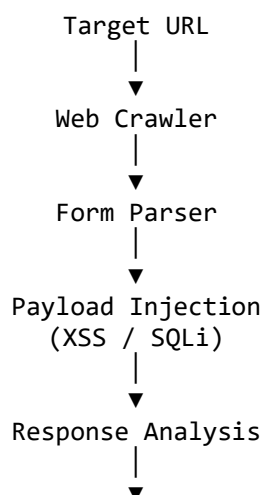
9.2 SQL Injection (SQLi)

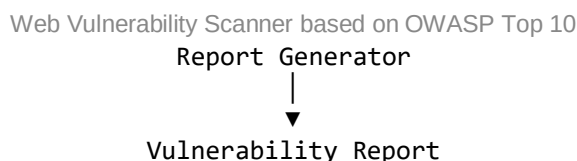
SQL Injection manipulates database queries.

Example:

```
' OR '1'='1
```

10. System Architecture





11. Technology Stack

Programming Language

- Python

Libraries

- Requests
- BeautifulSoup
- Flask

Tools

- VS Code
- Kali Linux
- GitHub

12. Expected Output

The scanner should detect:

- XSS vulnerabilities
- SQL Injection vulnerabilities

Generated Output:

```
[+] XSS Vulnerability Found [+] SQL Injection Found
```

13. Conclusion

The Web Vulnerability Scanner successfully demonstrates how automated security testing can identify common web application vulnerabilities. The project improves understanding of web security concepts and OWASP Top 10 risks while providing practical experience in cybersecurity and Python development.

14. Future Scope

- Add GUI Dashboard
- Support CSRF Detection
- Support File Inclusion Detection

- PDF Report Generation
- Multithreaded Scanning
- AI-based Vulnerability Analysis